

[C++ Programming]

2. Basic Syntax & Debugging

Contents

[전반적으로 C언어 복습단계!]

I. 배열

- 개요
- 선언과 초기화
- 접근

II. 명령문과 연산자

- 표현식
- 명령문
- 연산자 (대입, 산술, 증감, 관계, 논리, 멤버 접근, 기타)

III. 제어문

IV. 디버깅

I. 배열

- 배열
 - 복합(compound) 데이터 타입 (Class도 마찬가지!)
 - “동일한 타입”인 요소들의 집합!
- 개별 요소들에 직접적인 접근이 가능함!
- 사용 목적
 - 아래와 같은 코드를 효율적으로 사용 및 활용할 수 있음!
 - Ex) `int score_1 = 0;`
 - `int score_2 = 0`
 - `int score_3 = 0`
 - ...
 - `int score_1000 = 0`

- "고정된 길이" : 즉, 한번 정해진 길이는 변경할 수 없다!
- "연속된 메모리 주소에 저장"
- 인덱스를 통해 접근 가능
- 인덱스는 0부터 시작하며, 마지막 인덱스는 size - 1
- Out of bound 체크 하지 않음
- 초기화 필요
- 효율적인 데이터 구조

scores	index
100	[0]
85	[1]
21	[2]
56	[3]
70	[4]
95	[5]

길이가 6인 배열

```
cpp_basic (전역 범위) main
1  #include <iostream>
2
3  int main()
4  {
5      int scores[6] = { 100, 85, 21, 56, 70, 95 };
6
7      return 0;
8  }
```

메모리 1		
주소: &scores		
0x00000000D574FF5D8	64	d
0x00000000D574FF5D9	00	.
0x00000000D574FF5DA	00	.
0x00000000D574FF5DB	00	.
0x00000000D574FF5DC	55	U
0x00000000D574FF5DD	00	.
0x00000000D574FF5DE	00	.
0x00000000D574FF5DF	00	.
0x00000000D574FF5E0	15	.
0x00000000D574FF5E1	00	.
0x00000000D574FF5E2	00	.
0x00000000D574FF5E3	00	.
0x00000000D574FF5E4	38	8
0x00000000D574FF5E5	00	.
0x00000000D574FF5E6	00	.
0x00000000D574FF5E7	00	.
0x00000000D574FF5E8	46	F
0x00000000D574FF5E9	00	.
0x00000000D574FF5EA	00	.
0x00000000D574FF5EB	00	.
0x00000000D574FF5EC	5f	-
0x00000000D574FF5ED	00	.
0x00000000D574FF5EE	00	.
0x00000000D574FF5EF	00	.
0x00000000D574FF5F0	cc	?
0x00000000D574FF5F1	cc	?
0x00000000D574FF5F2	cc	?

계산기	
프로그래머	
64	
HEX 64	
DEC 100	
OCT 144	
BIN 0110 0100	
QWORD MS Mv	
비트 비트 시프트	
A << >> CE	
B () % ÷	
C 7 8 9 x	

- 배열의 정의
 - 기본형 : `int number[7]` → 요소타입 배열이름[요소 개수];
 - 이때, 요소 개수는 "상수"여야만 한다.

```
const int number = 7;  
int chance[number];  
.....
```

```
cpp_basic (전역 범위) main()  
1  #include <iostream>  
2  
3  using namespace std;  
4  
5  int main()  
6  {  
7      int scores[] = { 100, 85, 21, 56, 70, 95 };  
8  
9      cout << scores[0] << endl;  
10     scores[1] = 123;  
11     cout << scores[1] << endl;  
12  
13     return 0;  
14 }
```

```
Microsoft Visual Studio 디버그 × + v  
100  
123  
C:\Users\3388t\OneDrive\Desktop\MOBINN\mobinn
```

- 배열의 초기화

- `int my_array[] = { 1, 2, 3, 4, 5 }` 이런 식으로 automatic sizing array를 많이 사용한다!

```
1 int scores[5] = {100, 85, 21, 56, 70};  
2  
3 int highScores[10] = {3, 5}; //3, 5 and remaining 0  
4  
5 const int daysInYear = 365;  
6 double temperature[daysInYear] = {0}; //all to zero  
7  
8 int myArray[] = {1, 2, 3, 4, 5}; //automatic sizing
```

II. 명령문과 연산자

- 표현식

- 코드의 가장 작은 구성 요소
 - EX, 연산을 통해 값을 계산 / 리터럴, 변수 등등

```
1 34 // literal
2 favorite_numver //variable
3 3+5 // addition
4 3*5 // multiplication
5 a>b // relational
6 a = b // assignment
```

■ 명령문

- 명령을 수행하는 코드 단위
- 세미콜론(";")으로 끝나는 문장
- 표현식의 집합

```
1 int favorite_number; //declaration
2 favorite_number = 20; // assignment
3 3 + 5; // expression
4 favorite_number = 3 * 5; // assignment
5 if (a>b) cout << "a is greater than b"; //if
```

- 연산자
 - 단항(unary), 이항(binary), 삼항(ternary) 연산자
- 대입
 - 오른쪽의 값을 계산하여 왼쪽에 대입
 - 컴파일러가 대입이 가능하지 체크함
 - 왼쪽은 대입이 가능 해야함.

```
cpp_basic (전역)  
1 int num1 = 0;  
2 num1 = "cho";
```

- 산술 연산자
 - `+, -, *, /, %`

- 증감 연산자
 - Prefix (`++num`, `--num`) : 대입 전 증감
 - Postfix (`num++`, `num--`) : 대입 후 증감

- 비교 연산자
 - `==, !=`
 - 결과는 `true` or `false`로 나옴

- 관계 연산자
 - `<, >, <=, >=`

- 논리 연산자
 - `!, &&, ||` : not, and, or
 - Short-circuit evaluation
 - 결과 파악이 가능한 경우 나머지 연산을 하지 않음 (최적화)

III. 제어문

- 조건문
 - if-else문 (블록, nested)
 - Switch문
 - **?: 연산자**

 - 반복문
 - for문
 - while, do-while문
 - **continue, break**
- 놓쳤던 부분만 노트에 표시!

- **? : 연산자**
 - conditional_expr은 boolean 표현식
 - 표현식이 참이면, expr1의 값을 리턴
 - 표현식이 거짓이라면 expr2의 값을 리턴
 - If-else 문의 사용과 유사함
 - **삼항 연산자**

```
result = (a > b) ? a : b;
```

```
/*if (a > b)
{
    result = a;
}
else
{
    result = b;
}*/
```

```
cpp_basic (전역 범위)
1  #include <iostream>
2
3  int main()
4  {
5      int a = 9;
6      int b = 19;
7      int result = 0;
8
9      result = (a > b) ? a : b;
10     std::cout << result;
11 }
```

Microsoft Visual Studio 디버그 ×

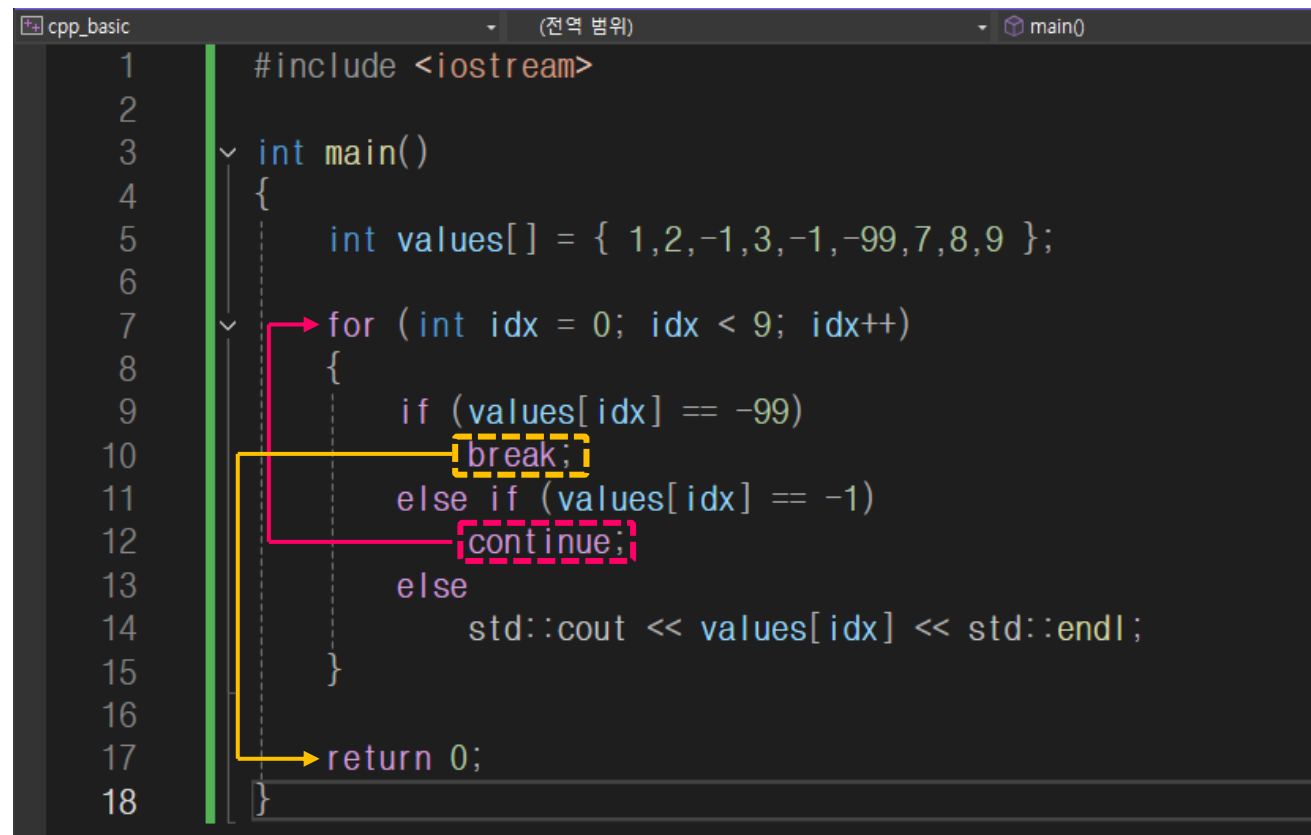
19
C:\Users\3388t\OneDrive\D
께 종료되었습니다

▪ continue

- Continue문 이후의 문장은 실행되지 않음
- 다음 iteration으로 곧바로 넘어가기 위해 사용
: 즉, 바로 다음 반복문이 실행됨!

▪ break

- Break문 이후의 문장은 실행되지 않음
- 루프 밖으로 바로 빠져나가기 위해 사용



```
1 #include <iostream>
2
3 int main()
4 {
5     int values[] = { 1,2,-1,3,-1,-99,7,8,9 };
6
7     for (int idx = 0; idx < 9; idx++)
8     {
9         if (values[idx] == -99)
10             break;
11         else if (values[idx] == -1)
12             continue;
13         else
14             std::cout << values[idx] << std::endl;
15     }
16
17     return 0;
18 }
```

- 배열 & 제어문 예제~

```
cpp_basic (전역 범위) main()
1  #include <iostream>
2
3  int main()
4  {
5      int scores[] = { 100, 99, 98 };
6
7      for (int idx = 0 ; idx < 3 ; idx++)
8      {
9          std::cout << scores[idx] << std::endl;
10     }
11
12     return 0;
13 }
```

```
Microsoft Visual Studio 디버그 ×
100
99
98
```

IV. 디버깅

▪ Debug

- 버그(오류)를 제거하는 것

- 오류 종류

- ✓ 컴파일 / 링크 오류 : 오류의 원인이 명확하며, 출력 창을 통해 쉽게 원인 파악 가능!

- ✓ 런타임 오류

- ✓ 논리적 오류

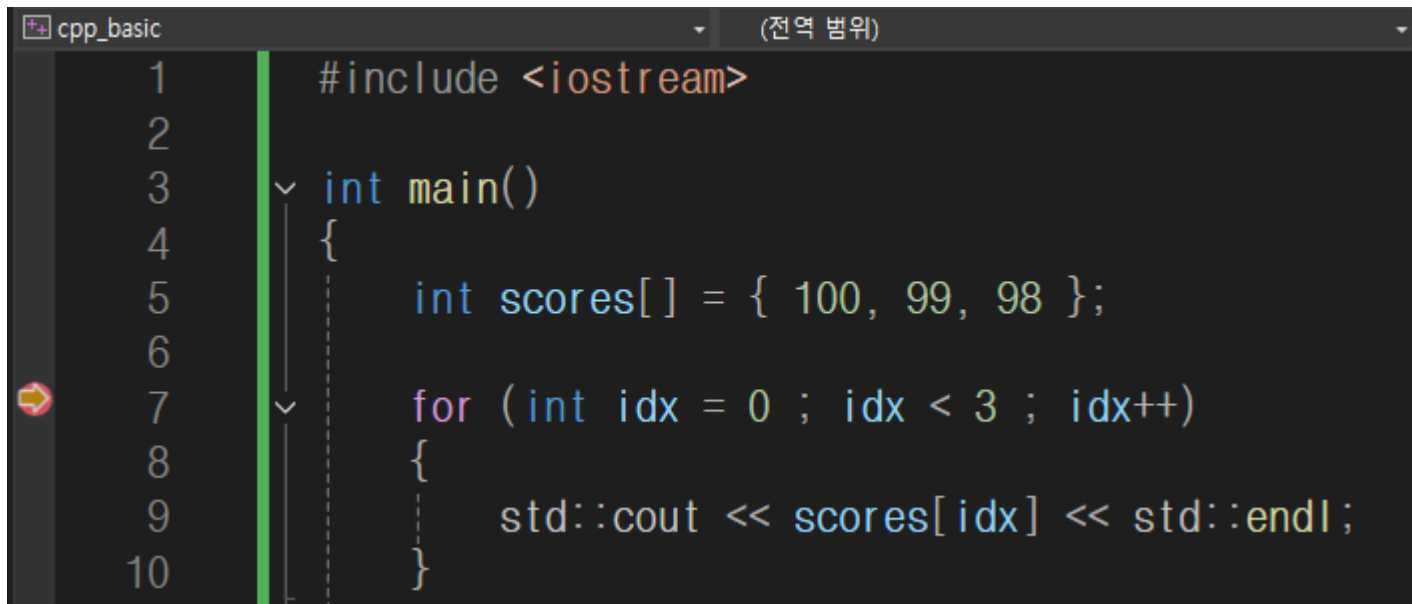
→ 어려운 오류.. IDE에서 제공하는 디버깅 기능과 테스트를 통해 현명히 차근차근 해결

▪ 중요한 점은... 개발 숙련도가 높아지더라도 오류 없는 프로그램을 단번에 개발하는 것은 불가능!

- 숙련도가 높아짐에 따라 오류의 "확률" 이 줄고, 디버깅 속도가 빨라지는 것!
- 즉, 오류 많이 내보고! 스스로 디버깅을 반복적으로 수행하고 고민하는 경험을 쌓아갈 것!

■ 중단점

- 임의의 위치에서 프로그램을 정지시킬 수 있는 점
 - ✓ **F5** : 다음 중단점까지 진행
 - ✓ **F9** : 중단점 생성 [마우스 클릭으로 생성됨]
 - ✓ **F10** : 해당 명령문 실행하고 다음 명령줄로 진행 (step over)
 - ✓ **F11** : 명령문 내에서 호출하는 추가 코드(ex.함수)로 들어감 (step into)
 - ✓ **Shift + F11** : 현재 명령문이 호출되는 상위 stack으로 나가기 (step out)



```
cpp_basic (전역 범위)
1  #include <iostream>
2
3  int main()
4  {
5      int scores[] = { 100, 99, 98 };
6
7      for (int idx = 0 ; idx < 3 ; idx++)
8      {
9          std::cout << scores[idx] << std::endl;
10     }
```

The screenshot shows a C++ code editor with a file named 'cpp_basic'. The code is as follows:

```
#include <iostream>

int main()
{
    int scores[] = { 100, 99, 98 };

    for (int idx = 0 ; idx < 3 ; idx++)
    {
        std::cout << scores[idx] << std::endl;
    }
```

A green vertical line is positioned at the start of line 7, indicating a break point. A red arrow icon is visible on the left margin next to line 7.

메모리 1 250919_4.cpp 250919_3.cpp 250919_2.cpp 250919_1.cpp 250907_1.cpp

cpp_basic (전역 범위) main()

```

1  #include <iostream>
2
3  int main()
4  {
5      int values[] = { 1,2,-1,3,-1,-99,7,8,9 };
6
7      for (int idx = 0; idx < 9; idx++)    경과 시간 1ms 이하
8      {
9          if (values[idx] == -99)
10             break;
11         else if (values[idx] == -1)
12             continue;
13         else
14             std::cout << values[idx] << std::endl;
15     }
16
17     return 0;
18 }
  
```

19 % 문제가 검색되지 않음 | 19 % 줄: 18 문자: 2 탭

검색 (Ctrl+E) | 검색 심도: 3 | [lab]

이름	값	형식
idx	1	int
values	0x004ff788 {1, 2, -1, 3, -1, -99, 7, 8, 9}	int[9]

C:\Users\W3388\OneDrive\W

디버깅(D) 테스트(S) 분석(N) 도구(T) 확장(X) 창(W) 도움말(H) 검색 cpp_basic

창(W) ▶ 중단점(B) Alt+F9

그래픽(C) ▶ 예외 설정(X) Ctrl+Alt+E

▶ 계속(C) F5

모두 중단(K) Ctrl+Alt+Break

▶ 디버깅 중지(E) Shift+F5

모두 분리(L)

모두 종료(M)

▶ 다시 시작(R) Ctrl+Shift+F5

▶ 코드 변경 내용 적용(A) Alt+F10

▶ 성능 프로파일러(F...) Alt+F2

▶ 성능 프로파일러 다시 시작(L) Shift+Alt+F2

▶ 프로세스에 연결(P...) Ctrl+Alt+P

▶ 디버깅 덤프 파일(M)

기타 디버깅 대상(H) ▶

▶ 한 단계씩 코드 실행(I) F11

▶ 프로시저 단위 실행(O) F10

▶ 프로시저 나가기(T) Shift+F11

66 ▶ 간략한 조사식(Q...) Shift+F9

▶ 중단점 설정/해제(G) F9

▶ 새 중단점(B)

▶ 모든 중단점 삭제(A) Ctrl+Shift+F9

▶ 모든 중단점 해제(N)

▶ 모든 DataTips 지우기(A)

▶ DataTips 내보내기(X)...

▶ DataTips 가져오기(I)...

▶ 다른 이름으로 덤프 저장(V)...

▶ 옵션(O)...

cpp_basic 디버깅 속성

조사식(W) ▶

▶ 자동(A) Ctrl+Alt+V, A

▶ 지역(L) Alt+4

▶ 직접 실행(I) Ctrl+Alt+I

▶ 호출 스택(C) Alt+7

▶ 스레드(H) Ctrl+Alt+H

▶ 모듈(O) Ctrl+Alt+U

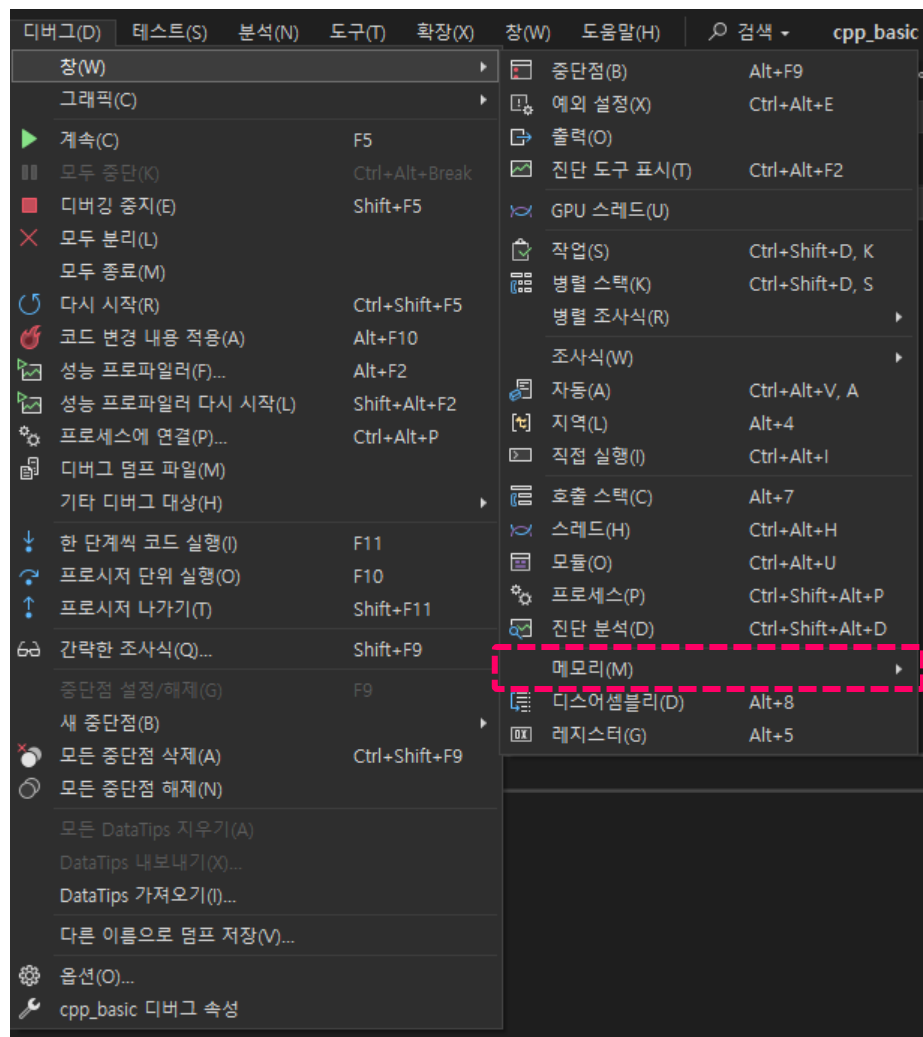
▶ 프로세스(P) Ctrl+Shift+Alt+P

▶ 진단 분석(D) Ctrl+Shift+Alt+D

▶ 메모리(M) ▶

▶ 디스어셈블리(D) Alt+8

▶ 레지스터(G) Alt+5



메모리 1		
주소: &scores		
0x0000000D574FF5D8	64	d
0x0000000D574FF5D9	00	.
0x0000000D574FF5DA	00	.
0x0000000D574FF5DB	00	.
0x0000000D574FF5DC	55	U
0x0000000D574FF5DD	00	.
0x0000000D574FF5DE	00	.
0x0000000D574FF5DF	00	.
0x0000000D574FF5E0	15	.
0x0000000D574FF5E1	00	.
0x0000000D574FF5E2	00	.
0x0000000D574FF5E3	00	.
0x0000000D574FF5E4	38	8
0x0000000D574FF5E5	00	.
0x0000000D574FF5E6	00	.
0x0000000D574FF5E7	00	.
0x0000000D574FF5E8	46	F
0x0000000D574FF5E9	00	.
0x0000000D574FF5EA	00	.
0x0000000D574FF5EB	00	.
0x0000000D574FF5EC	5f	—
0x0000000D574FF5ED	00	.
0x0000000D574FF5EE	00	.
0x0000000D574FF5EF	00	.
0x0000000D574FF5F0	cc	?
0x0000000D574FF5F1	cc	?
0x0000000D574FF5F2	cc	?